

Snake Game의 구현

22113821 박정은

1. 서론

2022학년도 2학기 C프로그래밍 학기말 프로젝트로 Snake Game을 구현하였다. Snake Game은 고전 게임 Snake에서 전반적인 아이디어를 얻어 제작된 것으로, Snake의 기능 일부를 그대로 사용하거나, 제거하고, 새로운 기능을 추가하여 재탄생했다. 제거된 기능은 snake의 Self Obstacle이고, 추가된 기능은 Vitality, Time Checking 등이다. 코드는 모두 C언어로 작성되었고, Visual Studio 2022 IDE에서 개발 환경에서 제작하였다.

게임 Snake Game 개발에 필요한 프로그래밍 지식을 2장 관련 연구에 기술하였고, 게임을 어떤 식으로 구현할 것인지에 대한 전반적인 아이디어를 이미지와 함께 3장 프로그램 설계에 기술하였다. 4장 프로그램 구현에는 아이디어를 실제 코드로 구현해 게임을 완성하는 과정을 담았다. 5장 프로그램의 동작과 결과에는 게임이 실행되고, 게임을 진행한 사용자가 얻을 수 있는 결과를 이미지와 함께 기술하였다. 6장 결과 분석 및 토의에서는 부족했던 부분, 어려웠던 부분들에 대해 기술, 7-9장은 순서대로 결론, 참고 문헌, 별첨을 기록했다.

2. 관련 연구

프로젝트에 추가된 헤더 파일은 총 5개로, <stdio.h>, <stdlib.h>, <conio.h>, <time.h>, <windows.h>다. 각각의 헤더파일에 대한 연구를 진행했다.

<stdio.h>는 표준 입출력 작업을 수행하는 함수들로 이루어진 헤더 파일이다. 게임 플레이어의 데이터를 프로그램이 종료된 이후에도 유지하고자 할 때, 헤더 파일에 정의된 fopen, fprintf 함수를 쓴다. fopen은 데이터를 저장할 파일을 특정 모드로 열고 파일 포인터를 반환한다. fprintf는 앞서 받은 포인터로 파일에 접근해 데이터

를 특정 형식으로 저장한다.

<stdlib.h>는 메모리 할당, 프로세스 제어, 문자열 변환 함수 등을 포함한 다목적 표준 라이브러리의 헤더 파일이다. 게임에서 객체의 랜덤 좌표를 생성하고자 할 때 자주 사용하는 함수는 헤더 파일에 정의된 rand다. rand는 0부터 RAND_MAX 사이의 숫자 하나를 반환한다. system 함수는 시스템 명령을 수행하는 함수다. 인자 값을 cmd 명령 문자열로 받고, 그 명령을 실행한다. 게임 화면을 초기화하고자 한다면, 명령 문자열로 cls를 입력하여, 즉, system("cls")를 작성하여 콘솔 화면을 초기화한다.

<conio.h>는 콘솔 입출력 함수를 제공한다. 헤더 파일에 포함된 멤버 함수는 kbhit, cgets, csanf, putch, cputs, cprintf, clrscr, getch이다. 이 중, 게임 캐릭터와 같은 객체의 위치를 조정할 때 자주 사용되는 함수는 kbhit와 getch다. kbhit는 keyboard hit의 약자로, 버퍼에 값이 존재하면 1을, 존재하지 않으면 0을 리턴한다. 즉, 사용자가 키보드를 입력하였는지 여부를 확인할 때 주로 이용된다. getch는 get character의 약자로, 사용자로부터 키보드의 키 하나를 입력받고, 그 키 값을 즉시 반환한다. 즉, 표준 입력 버퍼를 거치지 않는 콘솔 입력 함수다.

<time.h>는 날짜 및 시간 조작을 다루는 여러 함수를 제공한다. 포함된 멤버 함수는 clock, difftime, time, asctime, ctime, mktime, gmtime, localtime이다. 이 중, 한 사용자의 게임 진행 시간을 구하고자 할 때 주로 활용되는 함수는 clock과 time다. clock는 프로그램 시작 후 몇 클록이 지났는지를 계산해 그 값을 반환한다. 만약 값을 읽지 못했다면 -1을 반환한다. time은 협정 세계시 기준으로 1970년 1월 1일 0시 0분부터 현재까지의 시간을 초 단위로 반환한다.

<windows.h>는 윈도우 운영체제 전용 헤더 파일로, Windows API에 존재하는 모든 함수가 선언돼 있다. 헤더 파일에 선언된 Sleep 함수는 인자로 받은 시간 동안 현재의 스레드에서 다음 동작을 실행하지 않고 대기한다. 인자의 단위는 밀리초를 사용한다. 이 함수를 이용해 게임 실행을 잠시 멈추는 기능을 구현할 수 있다.

3. 프로그램 설계

Snake Game은 사용자가 Snake를 방향키로 컨트롤 해 최대한 오랫동안 생존하도록 하는 게임이다. 만약 Snake가 game field에서 벗어나게 되거나, Vitality 값이 0이 되면 게임은 자동으로 종료된다. Vitality 값은 정해진 시간이 지날 때마다 1씩 감소하는데, 레벨이 증가하면 감소 Speed 또한 빨라진다. 참고로 Vitality는 5로 초기화 돼 있다. Medicine을 획득하면 Vitality를 회복할 수 있다. 한 번 획득할 때마다 Vitality가 1씩 증가한다. 여기서 Snake는 사용자가 컨트롤 하는 대상의 이름, Vitality는 생명력이 어느 정도인지를 나타내는 지표, game field는 Snake가 이동할 수 있는 범위다.

프로젝트를 진행하면서, 기본적인 틀을 잡기 위해 콘솔 화면을 크게 4가지로 분류했다. 내용은 다음과 같다.

첫 번째는, 변수 gameStatus(게임 상태)의 값이 0일 때 나타나는 시작 화면이다. 프로그램을 실행하면 변수 gameStatus는 0으로 초기화되므로, 가장 먼저 보이는 콘솔 화면이 바로 시작 화면이다. 시작 화면에서는 게임명을 밝히고, 표준 입력 함수 scanf를 이용해 게임을 바로 진행할 것(1)인지, 도움말을 읽을 것(2)인지를 사용자가 선택할 수 있도록 한다. 이때 받은 값은 gameStatus에 대입된다.

두 번째는, gameStatus의 값이 2일 때 나타나는 도움말 화면이다. 도움말 화면은 게임 및 게임 제작자에 대해 간단히 소개하고, 게임을 할 때 꼭 알아야 할 기본적인 규칙을 설명한다. 게임을 시작할 수 있도록, scanf로 게임 시작에 필요한 값(1)을 사용자로부터 받는다.

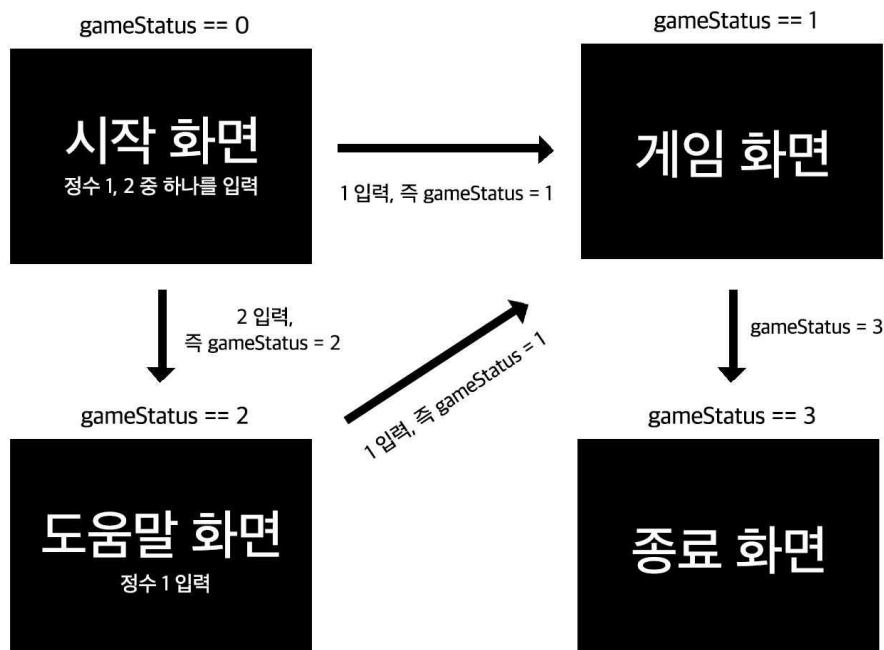
세 번째는, gameStatus의 값이 1일 때 나타나는 게임 화면이다. 게임 화면은 뱀이 자유롭게 움직일 수 있는 gamefield, 현재 사용자의 점수 및 최고 점수, Vitality가 표시된다.

네 번째는, gameStatus의 값이 3일 때 나타나는 종료 화면이다. 게임이 종료되면,

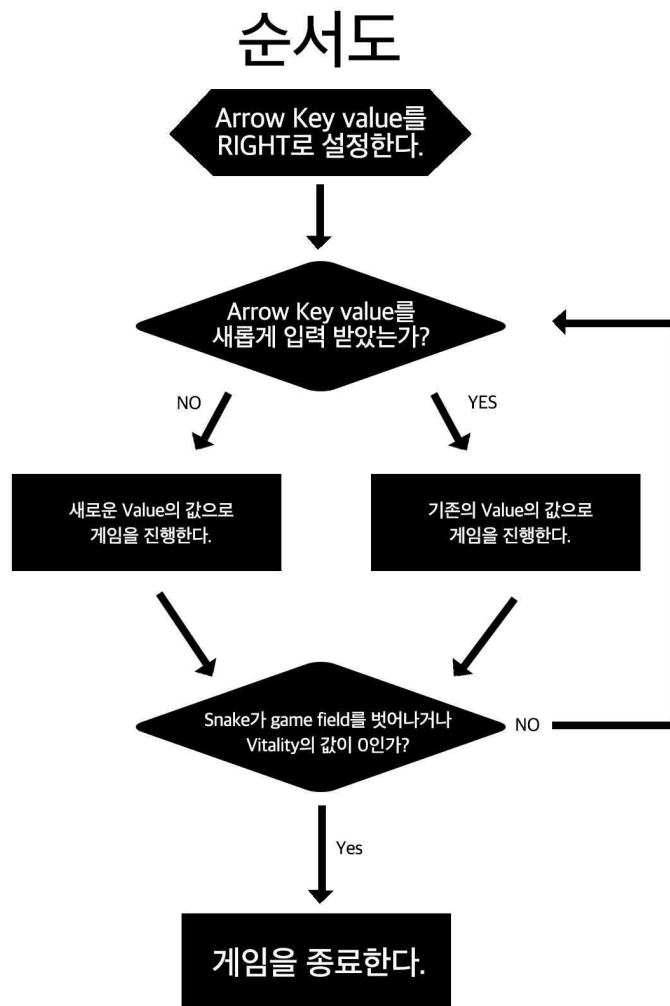
gameStatus의 값이 1에서 3으로 바뀐다. 종료 화면이 나타나면 더이상 게임을 진행할 수 없다. 게임이 종료됐다는 사실을 알리는 문구를 추가하고, 최고 점수를 갱신했을 경우, 그 점수로 갱신한다. 사용자의 최종 점수도 표시한다.

이를 시각화하여 나타내면 다음과 같다.

콘솔화면 구성



이 4가지 콘솔 화면 중 가장 핵심이 되는 게임 화면에서, 구현해야 할 알고리즘은 다음과 같다. 우선, Arrow key를 하나의 값으로 초기화하고, Arrow Key value의 입력 여부를 확인한다. 새로운 Value의 값이 들어올 경우, Snake를 해당 방향으로 이동시키고, 그렇지 않으면 기존의 방향 그대로 진행시킨다. 이동이 끝나면, Snake의 위치와 Vitality의 값을 파악한다. 만약 위치 값과 Vitality 값이 게임 진행 조건과 맞지 않을 경우 게임은 종료된다. 조건과 일치하면 Arrow key 값을 받는 단계로 다시 돌아간다. 이를 시각화한 순서도는 아래에 첨부하였다.



따라서, Snake Game에서 구현할 핵심적인 기능을 나열하면 다음과 같다.

- Snake와 game field 생성
- Arrow Key Value 입력 받기
- Snake 위치 업데이트 (화면 리셋 포함)
- 랜덤 Medicine 생성
- 흐른 시간 파악
- Speed 업데이트
- 최대 점수와 현재 점수 비교

4. 프로그램 구현

가장 먼저 게임 화면 콘솔에서 나타나는, Snake와 game field 기능을 구현하였다. 두 기능 모두 void print()로 정의한 함수 안에서 작성 되었다. 반복문 for을 이용하여 별표 모양의 제한 범위를 지정하였으며, 별표 모양의 가장자리를 제외한 부분은 Snake의 실제 행동 범위이므로 아스키 코드 값 32를 써서 space area로 지정하였다. 참고로 상수 ROW의 값은 40, COLUMN의 값은 20이다. Snake의 모양 또한 구현해야 한다. 따라서, gamefield 배열 속 값을 이용하여 0 이상, 5 미만일 경우 몸통, 5일 경우 머리를 출력하였다. 이때, 5는 고정된 Snake의 머리 값이다. 코드는 아래를 참조.

```
// printing out the field of play in the first row
printf("%c", 32);
for (i = 0; i <= COLUMN + 1; i++) {
    printf("%c", 42);
}
printf("\n");

// printing out the field of play in columns
for (i = 0; i < ROW; i++) {
    printf("%c", 32);
    printf("%c", 42);
    // SPACE AREA + SNAKE BODY AREA
    for (j = 0; j < COLUMN; j++) {
        if (gameField[i][j] == 0) {
            // SPACE AREA
            printf("%c", 32);
        }
        else if (gameField[i][j] > 0) {
            if (gameField[i][j] != Head) {
                // body part, not head
                printf("%c", 75);
            }
            else {
                // if field[i][j] == Head, Head part
                printf("%c", 64);
            }
        }
    }
    printf("%c", 42);
}

// printing out the field of play in the last row
printf("%c", 32);
for (i = 0; i <= COLUMN + 1; i++) {
    printf("%c", 42);
}
printf("\n");
```

그다음 Arrow Key value를 입력 받는 기능을 구현한다. 변수 direction에는 기존에 Snake가 이동하던 방향의 키 값이 담겨있다. 키보드 방향키로 값을 입력하지 않을 경우, 기존의 방향 그대로 진행된다. 값을 입력할 경우, direction에 새로운 방향의 키 값이 담기게 되고, 해당 방향으로 Snake가 이동한다. 이때, 이전의 키 값과 새로 입력 받은 키 값의 차이가 2거나 8이면 안 된다. 이때, RIGHT 방향키의 아스키 값은 77, RIGHT는 75, UP은 72, DOWN은 80이다. 즉, 서로 반대 방향의 키들

은 연속적으로 입력할 수 없다는 것이다. 이는, 게임의 난이도를 높이기 위해 특수적으로 구현한 것이다. 코드는 아래를 참조.

```
varKeyboard = getchReturnValue();
if (varKeyboard == -1000) {
    snakeMovement(direction);
}
else if (varKeyboard == 224) {
    varKeyboard = _getch();
    if (abs(direction - varKeyboard) != 2 && abs(direction - varKeyboard) != 8) {
        direction = varKeyboard;
        snakeMovement(direction);
    }
    else {
        snakeMovement(direction);
    }
}
```

새로운 방향키 값을 받았으면, Snake의 위치를 업데이트 해야 한다. 이에 대한 기능을 구현하기 위해 함수 void snakeMovement()를 선언하였다. 입력 받은 direction 값을 매개변수로 하여, 그 값이 RIGHT(상수, 77)에 해당하면 Snake의 y를 1 증가시킨다. 이후, 머리 값을 1 증가시키고, gameField 배열 중 머리가 될 원소의 값을 Head로 지정한다. 나머지 LEFT, UP, DOWN도 모두 같은 방식으로 진행한다. 단, 상하로 움직일 경우 y가 아닌 x가 변경된다. 일부 코드는 아래를 참조.

```
void snakeMovement(int direction) {
    switch (direction) {
        case RIGHT:
            // RIGHT
            y++;
            Head++;
            gameField[x][y] = Head;
            break;
    }
}
```

하지만, 콘솔창을 지속적으로 리셋하지 않으면 사용자가 방향키 값을 입력해 gameField 원소의 값을, 즉 Snake의 움직임을 변화시켰다 하더라도, 실제 화면에 그 결과가 나타나지는 않는다. 따라서 void ResetScreenPosition() 함수를 이용해 콘솔창을 지속적으로 리셋한다. 코드는 아래를 참조.

```

void ResetScreenPosition() {
    // Windows API functions
    HANDLE hOut;
    // HANDLE TYPE provides handle to the console of our program.
    COORD Position;
    // keeps the coordinate of the console screen.
    // COORD is a struct to hold screen coordinates X and Y
    hOut = GetStdHandle(STD_OUTPUT_HANDLE);
    Position.X = 0;
    Position.Y = 0;
    SetConsoleCursorPosition(hOut, Position);
    // Sets the cursor position in the specified console screen buffer.
}

```

사용자가 계속해서 게임을 이어나갈 수 있도록 랜덤 Medicine도 배치할 필요가 있다. 따라서 이에 대한 기능을 구현하기 위해 void randomMedicine() 함수를 정의한다. 무작위로 Medicine을 game field 내에 배치하는 것이 중요하므로, srand()와 rand() 함수를 이용해 무작위 값을 받고, medicine의 x좌표와 y좌표 각각 ROW와 COLUMN 상수에 해당하는 값으로 나눈다. 이때 나오는 값의 나머지가 실제 Medicine 좌표 값이 된다. gameField 배열 원소의 값이 -1일 경우, 즉 해당 배열 원소가 가지는 좌표에 Medicine이 생성되었을 경우, void print() 함수에서 아스키 코드 43에 해당하는 +모양이 표시된다.

```

void randomMedicine() {
    // place a medicine in random position
    srand(time(NULL));
    medicineX = rand() % ROW;
    // between 0 ~ 19
    medicineY = rand() % COLUMN;
    // between 0 ~ 39

    if (medicine == 0 && gameField[medicineX][medicineY] == 0) {
        // assign no medicine with empty space
        gameField[medicineX][medicineY] = -1;
        medicine = -1;
        // -1 means that medicine appears on screen
    }
}

```

Snake Game에서는 시간이 곧 Score를 의미하므로, 프로그램이 최초 실행되었을 때부터 현재까지 흐른 시간을 계산할 필요가 있다. clock() 함수를 이용해서 클럭

수를 반환하고, 그것을 CLOCKS_PER_SEC, 즉 1초 동안 동작하는 클럭 수로 나눠 흐른 시간을 계산한다. 코드는 아래를 참조.

```
currentTime = ((clock() - notPlayingTime) / CLOCKS_PER_SEC) - speedUpCount * 2;
```

Snake Game은 레벨을 정하고, 그에 따른 난도를 상이하게 하기 위해, Vitality가 감소하는 속도를 시간이 흐름에 따라 점차 빨라지게 한다. 이는 함수 timeChecking()을 선언하여 그 안에 구현하면 된다. 만약 currentTime, 즉 게임을 플레이한 시간이 30초보다 작을 경우 5초가 지날 때마다 Vitality가 1씩 감소하고, 나머지 레벨도 비슷한 방식으로 Vitality가 감소한다. 단, Vitality가 감소하는 속도가 4, 3, 2로 줄어든다.

```
void timeChecking() {
    if (currentTime < 30) {
        if (currentTime % 5 == 0 && currentTime != 0) {
            if (lastDivision != currentTime / 5) {
                --vitality;
                if (vitality == 0) {
                    gameOver(currentTime);
                }
                lastDivision = currentTime / 5;
            }
        }
    }

    else if (currentTime < 60) {
        if (currentTime % 4 == 0 && currentTime != 30) {
            if (lastDivision != currentTime / 4) {
                --vitality;
                if (vitality == 0) {
                    gameOver(currentTime);
                }
                lastDivision = currentTime / 4;
            }
        }
    }

    else if (currentTime < 90) {
        if (currentTime % 3 == 0 && currentTime != 60) {
            if (lastDivision != currentTime / 3) {
                --vitality;
                if (vitality == 0) {
                    gameOver(currentTime);
                }
                lastDivision = currentTime / 3;
            }
        }
    }

    else {
        if (currentTime % 2 == 0 && currentTime != 90) {
            if (lastDivision != currentTime / 2) {
                --vitality;
                if (vitality == 0) {
                    gameOver(currentTime);
                }
                lastDivision = currentTime / 2;
            }
        }
    }
}
```

게임이 끝났다면, 현재 자신의 Score, 즉 currentTime과 최대 점수를 비교할 필요가 있다. 최대 점수는 사용자 컴퓨터의 폴더 내 파일에 저장되어 있으므로, fopen() 함수로 값을 읽어온다. 읽어온 값은 fscanf() 함수로 highScore이란 변수에 담긴다. 그리고 highScore과 currentTime을 인자로 받은 매개변수 _t의 값을 비교하여, _t의 값이 더 크다고 판단되면 fopen(), fprintf() 함수를 사용해 신기록을 _t로 갱신한다.

```
f = fopen("highscore.txt", "r");
// file open with just read mode
fscanf(f, "%d", &highScore);
// scan the open file and assign the file txt to variable called highScore

printf("《 Your final score is: %d 》\n", _t);
if(_t > highScore) {
    printf("New Highest Score! Congrats :)\n", _t);
    f = fopen("highscore.txt", "w");
    fprintf(f, "%d", _t);
    // previous file text is erased whenever you print something different in file
    fclose(f);
}
```

이렇게 Snake Game에 필요한 중추적인 기능을 모두 구현하였다면, 그다음으로는 앞서 설명한 콘솔 화면 4가지를 구현한다. 이때, 사용자가 게임에 쉽게 접근할 수 있도록 시각적인 요소도 고려한다.

gameStatus가 1일 때, 즉 시작 화면에서 제목 Snake Game를 가독성 있게 표시하고, 1과 2 중 하나의 값을 입력 받을 수 있도록 scanf() 함수를 호출, 입력값을 gameStatus에 대입한다.

```

void startScreen() {
    printf("\n");
    printf("                ##\n");
    printf(" #####          ##\n");
    printf(" ##  ##          ##\n");
    printf(" ##          ## ##  #####  ##  ##  #####\n");
    printf(" #####  ##  ##  ##  ##  ##  ##  ##  ##\n");
    printf(" ##  ##  ##  ##  ##  ##  #####\n");
    printf(" ##  ##  ##  ##  ##  ##  ##  ##\n");
    printf(" #####  ##  ##  ##  ##  #####\n");

    printf("\n");

    printf(" #####\n");
    printf(" ##  ##\n");
    printf(" ##  #####  ##  ##  #####\n");
    printf(" ##  ##  ##  ##  ##  ##  ##\n");
    printf(" ##  ##  ##  ##  ##  ##  #####\n");
    printf(" ##  ##  ##  ##  ##  ##  ##\n");
    printf(" #####  ##  ##  ##  ##  #####\n");

    printf("\n");

    printf(" Press 1 to start\n");
    printf(" Press 2 to learn about Snake Game\n");

    printf(" >>> ");
    scanf("%d", &gameStatus);
    system("cls");
}

```

gameStatus가 2일 때, 즉 도움말 화면을 선언된 void howScreen() 함수 안에 구현한다. 필요한 정보를 출력한 뒤에는, scanf()로 입력을 받아 게임 시작을 할 수 있게 한다. 입력값은 물론 gameStatus에 저장된다.

```

void howScreen() {
    printf("\n");
    printf(" ##  ##          ##\n");
    printf(" ##  ##          ##\n");
    printf(" #####  #####  ##  ##  #####\n");
    printf(" ##  ##  ##  ##  ##  ##  ##\n");
    printf(" #####  ##  ##  ##  ##  ##  ##\n");
    printf(" ##  ##  ##  ##  ##  ##  ##\n");
    printf(" #####  #####  ##  ##  #####\n");

    printf("\n");

    printf(" ( 1 ) Snake Game is a game where snake needs to survive as much as it can eating a special medicine (x)\n");
    printf(" ( 2 ) Developed with ♥ By Park Jeong Eun, 22119821. Currently residing in Computer Software at SKU.\n");

    printf("\n");

    printf(" #####\n");
    printf(" ##  ##          ##\n");
    printf(" ##  ##  ##  ##  ##  ##  #####\n");
    printf(" #####  ##  ##  ##  ##  ##  ##\n");
    printf(" ##  ##  ##  ##  ##  ##  #####\n");
    printf(" ##  ##  ##  ##  ##  ##  ##\n");
    printf(" ##  ##  ##  ##  ##  ##  #####\n");

    printf("\n");

    printf(" ( 1 ) As time goes by, rate will be slower and lose vitality one by one.\n");
    printf("      When snake lose all the vitality, the game is automatically over.\n");
    printf(" ( 2 ) The speed of decreasing vitality rate will be getting faster each time the message pops up.\n");
    printf(" ( 3 ) When snake hit the wall(x), the game is automatically over.\n");
    printf(" ( 4 ) Your playing time is equivalent to your score and your score will be displayed.\n");
    printf(" ( 5 ) You're not allowed to use an opposite key while pressing a certain arrow key.\n");
    printf("      (i.e., when pressing the right arrow key, the left arrow key is inactive.)\n");

    printf("\n");

    printf(" ROWS: press 1 to start >>> ");

    while (1) {
        scanf("%d", &gameStatus);
        if (gameStatus == 1) break;
    }

    system("cls");
}

```

gameStatus가 1일 경우, 즉 게임 화면에서는 첫 번째 줄에 Vitality가 남은 정도를

나타낸다. Vitality가 5개일 경우, 속이 채워진 하트가 5개 출력되고, Vitality가 0개일 경우, 속이 채워지지 않은 하트가 5개 출력된다. currentTime, 즉 사용자의 Score도 플레이하는 도중 볼 수 있도록 최고 점수와 같이 출력한다. 이때 만약 플레이 도중 사용자의 점수가 기존 최고 점수를 넘어선 경우, 사용자의 점수가 최고 점수로 바뀌어 나타난다.

```
// printing out the vitality
for (i = 0; i < COLUMN - 23; i++) {
    printf("%c", 32);
}
printf(" [ Vitality : ");
for (i = 0; i < vitality; i++) {
    printf("♥");
}
for (i = 0; i < 5 - vitality; i++) {
    printf("♡");
}
printf(" ] \n");

// SCORE TABLE
printf(" Current Score : %3d | Highest Score : %3d\n", currentTime, currentTime > highScore ? currentTime : highScore);
```

게임이 끝날 경우, Game Over란 문구를 크게 화면에 표시하고, 아무 키나 눌러 게임을 종료시킬 수 있도록 한다. 함수가 종료되기 전 gameStatus 변수에 3을 대입 해주면 시스템이 종료된다.

```
void gameOver(clock_t _t) {
    printf("\n");
    system("cls");
    // remove all the previously printed messages

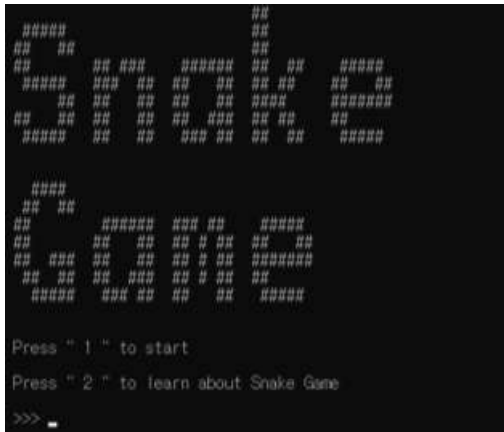
    printf("\n");
    printf("#####");
    printf(" 30 30\n");
    printf(" 30 ##### 30 30 #####\n");
    printf(" 30 30 30 30 30 30 30 30 30 30 30 30\n");
    printf(" 30 30 30 30 30 30 30 30 30 30 30 30\n");
    printf(" 30 30 30 30 30 30 30 30 30 30\n");
    printf(" ##### 30 30 30 30 30 30 30 30\n");

    printf(" #####");
    printf(" 30 30\n");
    printf(" 30 30 30 30 ##### 30 30\n");
    printf(" 30 30 30 30 30 30 30 30\n");
    printf(" 30 30 30 30 30 30 30 30\n");
    printf(" 30 30 30 30 30 30 30 30\n");
    printf(" 30 30 30 30 30 30 30 30\n");

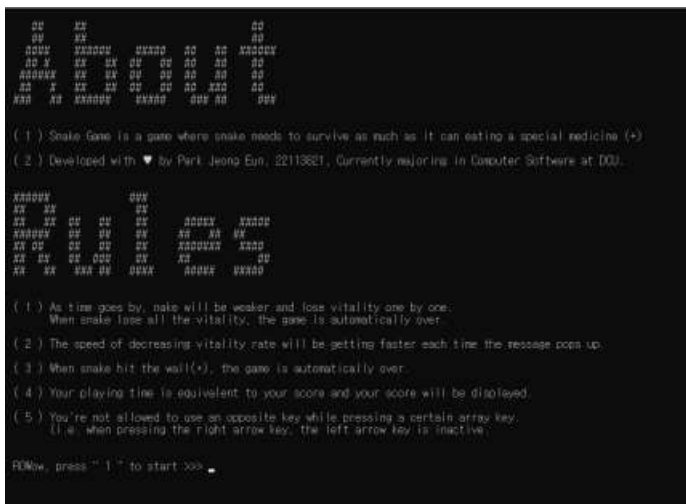
    printf(" [ Your final score is: %d ] \n", _t);
    if (_t > highScore) {
        printf(" New Highest Score! Congrats! \n", _t);
        F = fopen("highscore.txt", "w");
        fprintf(F, "%d", _t);
        // previous file text is erased whenever you print something different in file
        fclose(F);
    }
    printf("\n Enter any key to end game >>> ");
    scanf("%c", &gameStatus);
    gameStatus = 3;
}
```

5. 프로그램의 동작과 결과

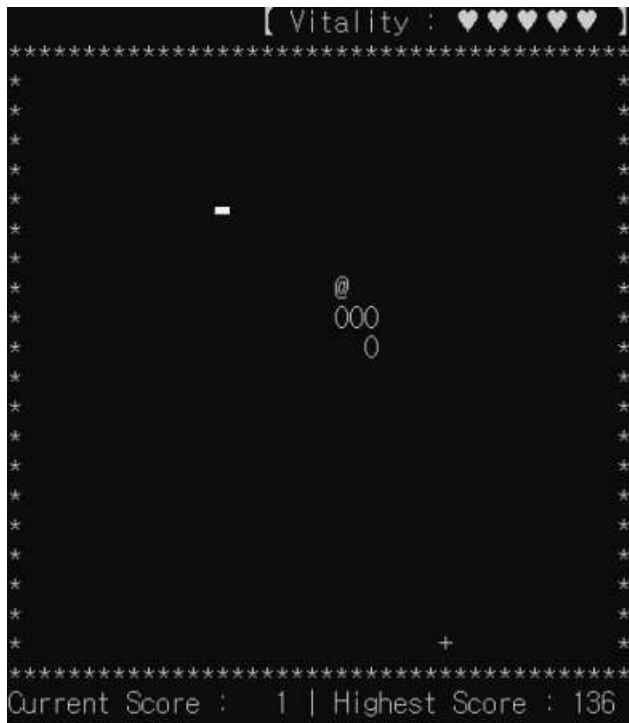
프로그램을 처음 실행했을 때, 앞서 설명한 시작 화면이 뜨고, 값 입력을 받는다.



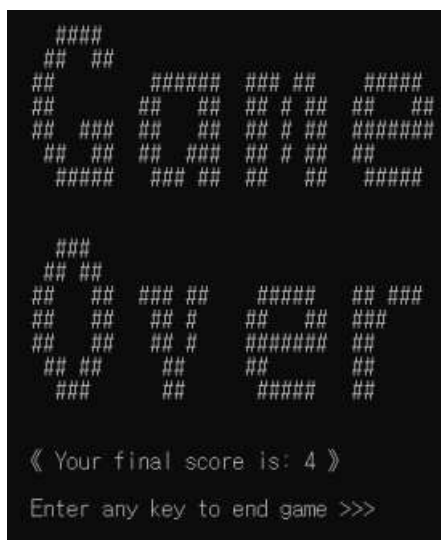
도움말 화면은 다음과 같이 구성돼 있다. 값 입력을 받는다.



게임 화면은 다음과 같다. Vitality, Current Score, Highest Score이 기록된다.



종료 화면은 다음과 같다. 신기록을 경신하지 못하였기 때문에 최종 점수만이 출력된다.



6. 결과 분석 및 토의

Timechecking 함수의 경우, while(1) 반복문 안에서 존재하기 때문에 gameStatus가 3이 되지 않는 이상 계속 실행되게 되는데, 이때 반복문을 한 번 실행하는 데

걸리는 시간이 1초보다 짧기 때문에 Vitality가 1초에 1개 감소하는 것이 아닌, 3-4개씩 감소한다는 문제가 있었다. 이를 해결하기 위해 lastDivision이란 변수를 추가하여, 정해진 초당 Vitality 감소가 한 번만 일어날 수 있게 하였다.

프로그램을 종료하지 않은 상태에서 게임을 재시작하는 기능 또한 포함하고자 하였으나, 변수가 이미 모두 변경돼 있는 상황에서 새로운 값으로 초기화한다는 것은 쉽지 않았다. 무엇보다 currentTime, 즉 프로그램이 시작되고 현재까지의 클럭 수를 이용해 얻어낸 플레이 타임은 클럭 수를 초기화해야 Score 또한 초기화가 되는데, 여러 검색을 진행한 후에도 클럭 수를 초기화하는 방법을 알아내지는 못했다. 물론 게임 횟수와 이전 플레이 시간을 담는 변수를 선언해서 currentTime을 초기화하는 건 가능하지만, 이렇게 되면 여러 변수와 예기치 못한 오류가 발생하기 때문에 따로 구현하지 않았다.

7. 결론

Snake Game 구현 프로젝트를 직접 진행했고, 원하는 기능을 3/4 수준으로 구현하는 데 성공했다. 굉장히 간단해 보이는 게임에도 수백, 수천 줄의 코드와 그에 대응하는 알고리즘이 필요하다는 사실을 다시 한 번 상기하게 되는 시간이었다. C언어는 정적인, 딱딱한 언어에 그치지 않는다는 나의 생각이 360도 바뀌게 되어 기쁘다. 그리고, 학기말 프로젝트를 진행하지 않았다면 써볼 일 없었을 여러 함수(i.e. getStdHandle())도 실습하듯 쓸 수 있었기에 좋은 경험이었다. 하지만, 유니 코드 출력 방법, 프로그램 종료 없이 재시작 방법은 여전히 알지 못하기 때문에 이 두 가지 부분에 대한 추가적인 학습이 필요할 것 같다. 2학년 때는 C언어가 커리큘럼에 포함되어 있지 않지만, 그래도 안주하지 않고 꾸준히 스스로 학습을 진행할 것이다.

8. 참고 문헌

<https://ko.wikipedia.org/wiki/Stdlib.h>

<https://individuality.tistory.com/55>

<https://m.blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=tipsware&logNo=221242072724>

https://en.wikipedia.org/wiki/C_date_and_time_functions
<https://modoocode.com/110>
<https://futurecreator.github.io/2018/06/07/computer-system-time/>
<https://en.wikipedia.org/wiki/Windows.h>
<https://www.geeksforgeeks.org/sleep-function-in-c/>
<https://www.geeksforgeeks.org/snake-game-in-c/>

9. 별첨 (소스코드와 주석)

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <time.h>
#include <windows.h>

/* 뱀 게임. 생명력이 줄지 않도록 계속해서 medicine를 먹어야 한다.
medicine를 먹지 않으면 특정 시간이 지날 때마다 생명력이 하나씩 줄어든다.
*/

#define ROW 20
// N은 Number of rows 행의 수가 20
#define COLUMN 40
// M은 Number of columns 열의 수가 40

#define RIGHT 77
#define LEFT 75
#define UP 72
#define DOWN 80

int i, j;
// two global variables for counting things
int gameField[ROW][COLUMN] = { 0 };
// initialize all the field array value to 0
int x, y;
// initialize the coordinate points for snake
int Gy, Head, Tail;
int gameStatus;
// check if game is working or not
int medicine = 0;
// initialize the medicine value to 0, 1 means it appears
int medicineX, medicineY;
// coordinate points for medicine
int varKeyboard;
// holds an input from the keyboard
int direction;
// snake-moving direction, whenever key pressed -> called
int vitality, count, speedUpCount = 0, lastDivision;
int highScore;
int lastDivisionSpeed = 0;
clock_t currentTime, start, end, notPlayingTime;

FILE *f;
// file handler

void snakeInitialization() {
    f = fopen("highscore.txt", "r");
    // file open with just read mode
    fscanf(f, "%d", &highScore);
    // scan the open file and assign the file txt to variable called highScore
    fclose(f);
}
```



```

// make sure to close file after finishing scanning the file

x = ROW / 2;
y = COLUMN / 2;
// begining points for snake, head of snake

Gy = y;
// initial y coordniate point
Head = 5;
// When array value holds the value 5, It's Head
Tail = 1;
// When array value holds the value 1, It's Tail
gameStatus = 0;
// set the Game value to 0, which is stopping. 1 -> running
direction = RIGHT;
// initialize the direction to where snake moves
vitality = 5;
count = 0, lastDivision = 0;

// setting the snake body (Head+Tail) part
for (i = 0; i < Head; i++) {
    Gy++;
    // 21, 22, 23, 24, 25
    gameField[x][Gy - Head] = i + 1;
    // initialize the y coordinate points Gy - Head (16, 17, 18, 19, 20)
    // to 1, 2, 3, 4, 5
}
}

void startScreen() {
    printf("\n");
    printf("                ##\n");
    printf("    #####                ##\n");
    printf("    ##    ##                ##\n");
    printf("    ##    ##    ##    ##    ##    ##    ##    ##    ##\n");
    printf("    #####    ##    ##    ##    ##    ##    ##    ##    ##\n");
    printf("    ##    ##    ##    ##    ##    ##    ##    ##    ##\n");
    printf("    ##    ##    ##    ##    ##    ##    ##    ##    ##\n");
    printf("    #####    ##    ##    ##    ##    ##    ##    ##    ##\n");

    printf("\n\n");

    printf("    ####\n");
    printf("    ##    ##\n");
    printf("    ##    #####    ##    ##    #####\n");
    printf("    ##    ##    ##    ##    ##    ##    ##    ##\n");
    printf("    ##    ##    ##    ##    ##    ##    #####\n");
    printf("    ##    ##    ##    ##    ##    ##    ##\n");
    printf("    #####    ##    ##    ##    ##    #####\n");

    printf("\n\n");

    printf("    Press \" 1 \" to start\n\n");
    printf("    Press \" 2 \" to learn about Snake Game\n\n");

    printf("    >>> ");
    scanf(" %d", &gameStatus);
    system("cls");
}

void howScreen() {
    printf("\n");
    printf("    ##    ##                ##\n");
    printf("    ##    ##                ##\n");
    printf("    #####    #####    ##    ##    #####\n");
    printf("    ##    ##    ##    ##    ##    ##    ##\n");
    printf("    #####    ##    ##    ##    ##    ##    ##\n");
    printf("    ##    ##    ##    ##    ##    ##    ##\n");

```

```

printf(" ###  ## #####  #####  ### ##  ###\n");

printf("\n\n");

printf(" ( 1 ) Snake Game is a game where snake needs to survive as much as it can eating
a special medicine (+)\n\n");

printf(" ( 2 ) Developed with ♥ by Park Jeong Eun, 22113821, Currently majoring in Computer
Software at DCU.\n");

printf("\n\n");

printf(" #####          ##\n");
printf(" ##  ##          ##\n");
printf(" ##  ##  ##  ##  ##  ##  #####  #####\n");
printf(" #####  ##  ##  ##  ##  ##  ##\n");
printf(" ##  ##  ##  ##  ##  #####  #####\n");
printf(" ##  ##  ##  ##  ##  ##  ##\n");
printf(" ##  ##  ##  ##  ##  ##  #####  #####\n");

printf("\n\n");

printf(" ( 1 ) As time goes by, nake will be weaker and lose vitality one by one.\n");
printf("      When snake lose all the vitality, the game is automatically over.\n\n");
printf(" ( 2 ) The speed of decreasing vitality rate will be getting faster each time the
message pops up.\n\n");
printf(" ( 3 ) When snake hit the wall(*), the game is automatically over.\n\n");
printf(" ( 4 ) Your playing time is equivalent to your score and your score will be
displayed.\n\n");
printf(" ( 5 ) You're not allowed to use an opposite key while pressing a certain array
key.\n");
printf("      (i.e. when pressing the right arrow key, the left arrow key is inactive.\n");

printf("\n\n ROWow, press \" 1 \" to start >>> ");

while (1) {
    scanf(" %d", &gameStatus);
    if (gameStatus == 1) break;
}

system("cls");
}

void print() {
    // printing out the vitality
    for (i = 0; i < COLUMN - 23; i++) {
        printf("%c", 32);
    }
    printf("【 Vitality : ");
    for (i = 0; i < vitality; i++) {
        printf("♥");
    }
    for (i = 0; i < 5 - vitality; i++) {
        printf("♡");
    }
    printf(" 】\n");

    // printing out the field of play in the first row
    printf("%c", 32);
    for (i = 0; i <= COLUMN + 1; i++) {
        printf("%c", 42);
    }
    printf("\n");

    // printing out the field of play in columns
    for (i = 0; i < ROW; i++) {
        printf("%c", 32);
        printf("%c", 42);
    }
}

```

```

// SPACE AREA + SNAKE BODY AREA
for (j = 0; j < COLUMN; j++) {
    if (gameField[i][j] == 0) {
        // SPACE AREA
        printf("%c", 32);
    }
    else if (gameField[i][j] > 0) {
        if (gameField[i][j] != Head) {
            // body part, not head
            printf("%c", 79);
        }
        else {
            // if field[i][j] == Head, Head part
            printf("%c", 64);
        }
    }
    else {
        // field array value less than 0
        if (gameField[i][j] == -1) {
            // if field array value is -1, medicine part
            printf("%c", 43);
        }
    }
}
printf("%c\n", 42);
}

// printing out the field of play in the last row
printf("%c", 32);
for (i = 0; i <= COLUMN + 1; i++) {
    printf("%c", 42);
}
printf("\n");

// SCORE TABLE
printf(" Current Score : %3d | Highest Score : %3d\n", currentTime, currentTime > highScore
? currentTime : highScore);
}

void speedManage() {
    if (currentTime % 30 == 0 && currentTime > 0 && currentTime <= 90) {
        if (lastDivisionSpeed != currentTime / 30) {
            lastDivisionSpeed = currentTime / 30;
            system("cls");
            // remove all the printed messages

            // speed up message
            printf("\n
            #####
            ##
            ##      #####      #####      #####      #####\n");
            printf(" #####  ##  ##  ##  ##  ##  ##  ##\n");
            printf("      ##  ##  ##  #####  #####  ##  ##\n");
            printf(" ##  ##  ##  ##  ##  ##  ##  ##\n");
            printf(" #####  #####  #####  #####  #####\n");
            printf("      ##\n");
            printf("      ##\n\n");
            printf("      ##\n");
            printf(" ##  ##  #####  ##\n");
            printf(" ##  ##  ##  ##  ##\n");
            printf(" ##  ##  ##  ##  ##\n");
            printf(" ##  ##  #####  ##\n");
            printf(" ##  ##  ##  ##\n");
            printf(" ##  ##  ##\n");
            printf(" #####  ##  ##\n");

            Sleep(2000);
            speedUpCount++;

```

```

        vitality = 5;

        system("cls");
        // set the vitality to 5, RESET
    }
}

void ResetScreenPosition() {
    // WIndows API functions
    HANDLE hOut;
    // HANDLE TYPE provides handle to the console of our program.
    COORD Position;
    // keeps the coordinate of the console screen.
    // COORD is a structrue to hold screen coordinates X and Y
    hOut = GetStdHandle(STD_OUTPUT_HANDLE);
    Position.X = 0;
    Position.Y = 0;
    SetConsoleCursorPosition(hOut, Position);
    // Sets the cursor position in the specified console screen buffer.
}

void randomMedicine(){
    // place a medicine in random position
    srand(time(NULL));
    medicineX = rand() % ROW;
    // between 0 ~ 19
    medicineY = rand() % COLUMN;
    // between 0 ~ 39

    if (medicine == 0 && gameField[medicineX][medicineY] == 0) {
        // assign no medicine with empty space
        gameField[medicineX][medicineY] = -1;
        medicine = -1;
        // -1 means that medicine appears on screen
    }
}

void gameOver(clock_t _t) {
    printf("\a");
    system("cls");
    // removel all the previously printed messages

    printf("\n");
    printf("    ###\n");
    printf("  ##  ##\n");
    printf("  ##      #####  ###  ##      #####\n");
    printf("  ##      ##  ##  ##  ##  ##  ##  ##\n");
    printf("  ##  ###  ##  ##  ##  ##  ##  #####\n");
    printf("  ##  ##  ##  ##  ##  ##  ##  ##\n");
    printf("  #####  ###  ##  ##  ##  #####\n\n");

    printf("    ##\n");
    printf("  ##  ##\n");
    printf("  ##  ##  ###  ##      #####  ##  ##\n");
    printf("  ##  ##  ##  #  ##  ##  ##  ##\n");
    printf("  ##  ##  ##  #  #####  ##\n");
    printf("  ##  ##      ##  ##      ##\n");
    printf("  ##      ##      #####  ##\n\n");

    printf("《 Your final score is: %d 》\n", _t);
    if(_t > highScore) {
        printf("  New Highest Score!  Congrats :)\n", _t);
        f = fopen("highscore.txt", "w");
        fprintf(f, "%d", _t);
        // previous file text is erased whenever you print something diffrent in file
        fclose(f);
    }
}

```

```

        printf("\n Enter any key to end game >>> ");
        scanf("");
        scanf(" %d", &gameStatus);
        gameStatus = 3;
    }

    int getchReturnValue() {
        if (_kbhit()) {
            // if keyboard is hit
            return _getch();
            // _getch() uses console input without input buffer
        }
        else {
            return -1000;
        }
    }

    void snakeMovement(int direction) {
        switch (direction) {
            case RIGHT:
                // RIGHT
                y++;
                if (y == COLUMN) {
                    gameOver(currentTime);
                }
                else if (gameField[x][y] == -1) {
                    // eats a medicine
                    medicine = 0;
                    if (vitality != 5) vitality++;
                }
                Head++;
                gameField[x][y] = Head;
                break;

            case LEFT:
                // LEFT
                y--;
                if (y == -1) {
                    gameOver(currentTime);
                }
                else if (gameField[x][y] == -1) {
                    // eats a medicine
                    medicine = 0;
                    if (vitality != 5) vitality++;
                }
                Head++;
                gameField[x][y] = Head;
                break;

            case UP:
                // UP
                x--;
                if (x == -1) {
                    gameOver(currentTime);
                }
                else if (gameField[x][y] == -1) {
                    // eats a medicine
                    medicine = 0;
                    if (vitality != 5) {
                        vitality++;
                    }
                }
                Head++;
                gameField[x][y] = Head;
                break;

            case DOWN:

```

```

        // DOWN
        x++;
        if (x == ROW) {
            gameOver(currentTime);
        }
        else if (gameField[x][y] == -1) {
            // eats a medicine
            medicine = 0;
            if (vitality != 5) {
                vitality++;
            }
        }
        Head++;
        gameField[x][y] = Head;
        break;
    }

    for (i = 0; i < ROW; i++) {
        for (j = 0; j < COLUMN; j++) {
            if (gameField[i][j] == Tail) {
                gameField[i][j] = 0;
            }
        }
    }

    Tail++;
}

void timeChecking() {
    if (currentTime < 30) {
        if (currentTime % 5 == 0 && currentTime != 0) {
            if (lastDivision != currentTime / 5) {
                --vitality;
                if (vitality == 0) {
                    gameOver(currentTime);
                }
                lastDivision = currentTime / 5;
            }
        }
    }

    else if (currentTime < 60) {
        if (currentTime % 4 == 0 && currentTime != 30) {
            if (lastDivision != currentTime / 4) {
                --vitality;
                if (vitality == 0) {
                    gameOver(currentTime);
                }
                lastDivision = currentTime / 4;
            }
        }
    }

    else if (currentTime < 90) {
        if (currentTime % 3 == 0 && currentTime != 60) {
            if (lastDivision != currentTime / 3) {
                --vitality;
                if (vitality == 0) {
                    gameOver(currentTime);
                }
                lastDivision = currentTime / 3;
            }
        }
    }

    else {
        if (currentTime % 2 == 0 && currentTime != 90) {
            if (lastDivision != currentTime / 2) {

```

```

        --vitality;
        if (vitality == 0) {
            gameOver(currentTime);
        }
        lastDivision = currentTime / 2;
    }
}

int main() {
    start = clock();

    snakeInitialization();
    startScreen();

    if (gameStatus == 2) {
        howScreen();
    }

    end = clock();
    notPlayingTime = end - start;

    while (gameStatus == 1) {
        // if game value is 0, the game ends.
        currentTime = ((clock() - notPlayingTime) / CLOCKS_PER_SEC) - speedUpCount * 2;
        // continuously checking the current time
        print();
        speedManage();
        ResetScreenPosition();
        randomMedicine();

        varKeyboard = getchReturnValue();
        if (varKeyboard == -1000) {
            snakeMovement(direction);
        }
        else if (varKeyboard == 224) {
            varKeyboard = _getch();
            if (abs(direction - varKeyboard) != 2 && abs(direction - varKeyboard) != 8)
            {
                direction = varKeyboard;
                snakeMovement(direction);
            }
            else {
                snakeMovement(direction);
            }
        }
        timeChecking();
        Sleep(20);
    }
}

```